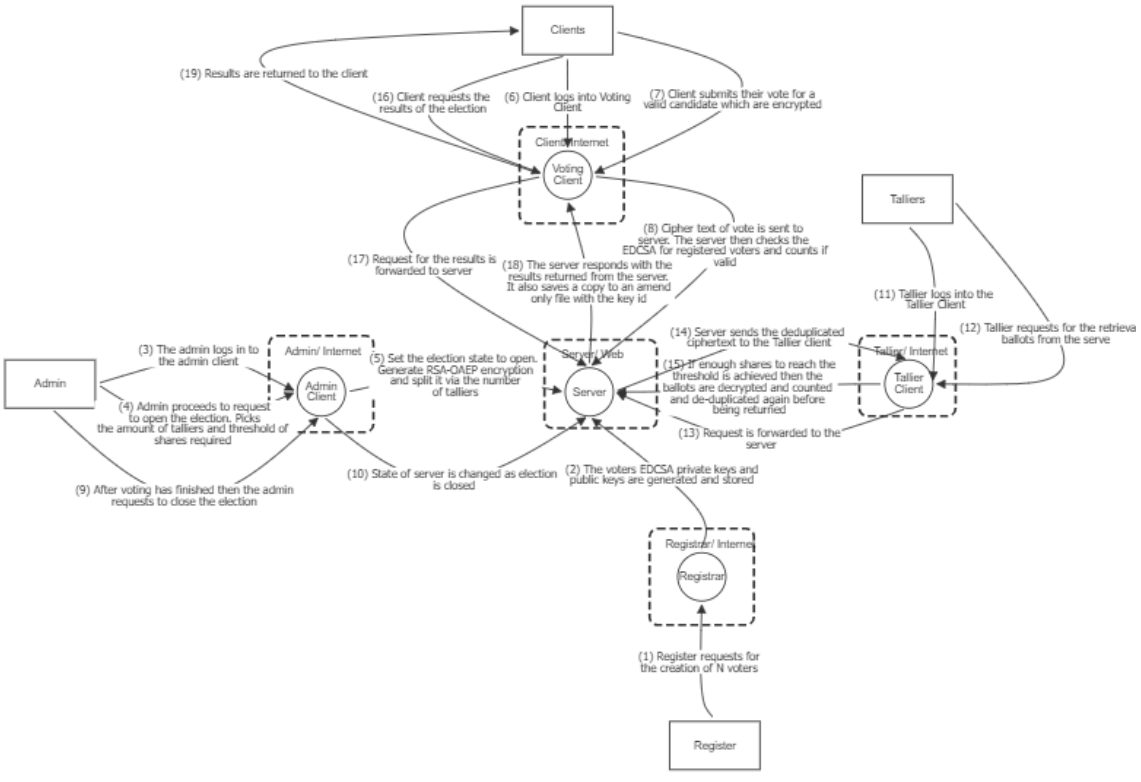
Secure Voting System Report

Assumptions

- Election key secrets are stored securely, and the shares are distributed off band to the talliers with uniquely identifiable random strings.
- Signature keys for the voters' private PEMs are securely stored and delivered off band to the voters, and the public keys are stored securely for the client/server.
- Authorisation for voters, admins and talliers are set up separately from the voting system with their usernames and passwords being chosen by the users with password policies in place. This is why the auth.py file is not used in the data flow diagram as it is purely for the test suite. Voter usernames are to be a random uniquely identifiable string. Expiry time of tokens also set lower just set as is for ease of testing.
- Registrar is given a register of all valid voters so it can register them. This register is held in a secure system and verification of voters occurs before they are added. This is why there is no logging in as it is configured from a predefined list of voters. Due to no logging in and no token being generated for the registrar /voters is not gated.

Data Flow Diagram



Architecture Overview

- admin.py: Uses requests, cryptography, shamir, pathlib, random and json. Acts as the election controller. Logs in as admin, generated RSA election keys, splits the private key using Shamir

secret sharing, writes the shares and the RSA primes, picks the threshold for reconstruction, sends the public key to the server and opens/ closes the election.

- client.py: Uses requests, cryptography, pathlib and json. Acts as the voting client used by the voters. Logs in, fetches candidates and the RSA public key, checks election is open, signs vote with ECDSA, encrypts vote with RSA-OAEP, checks candidate against server candidate list, submits ballots and retrieves results.

- registrar.py: Uses cryptography, requests, json, pathlib and uuid. Acts as the voter registration authority. Generates ECDSA keypairs for voters, stores private keys/public keys and queries the server for registered voters.

- server.py: Uses http.server, cryptography, logging, threading, kerberos, pathlib, hashlib, hmac, json and time. Acts as the central election server handling authentication, voting, pseudonym generation, ballot storage and result publication. Exposes endpoints behind role-based access controls, verifies tokens and signatures, enforces one vote per voter ID and election key ID, stores encrypted ballots, pseudonymises them for the taller and logs all actions.

- shamir.py: Uses random, typing and secrets. Implements Shamir secret sharing cryptography. Splits the RSA private keys into n shares and reconstructs when threshold of shares is reached.

- tallier.py: Uses requests, cryptography, shamir, pathlib, glob, typing and json. Acts as the vote decrypted and counter. Reconstructs the RSA private key from the Shamir shares, rejects insufficient or incorrect sets, fetches anonymised ballots, decrypts votes, tallies results ignoring duplicates/ incorrect votes and submits final count to server.

- kerberos.py: Uses hashlib, hmac, json, os, pathlib, typing and time. Provides authentication and token security for the system. Hashes passwords with PBKDF2, creates and verifies HMAC signed tokens and manages credentials.

- auth.py: Uses json, pathlib and kerberos. Acts as a user credential creator. Creates credentials for admin, tallier and voters that the server later verifies.

- test_script.py: Uses subprocess, requests, auth, json, sys, pathlib and time. End to end test suite for the system. Launches every component, simulates registration, voting, examples of election controls working, tallying and tests correctness.

Component interaction

- Registrar -> Server: Registrar generates voter ECDSA keypairs and the server only trusts vote signatures that match these public keys.

- Admin -> Server: Admin generates RSA election key and posts the public key to the server and opens/ closes the election

- Client -> Server: Client fetches the current election key and candidates, signs and encrypts a vote and submits voter id, ciphertext, signature and key id to the server. The server only accepts votes when the election is open and the signature gets verified.

- Tallier -> Server: Tallier runs after the election closes, reconstructs the RSA private key from shares when over threshold, decrypts and tallies the ballots, removes duplicates and submits the results to the server.

- Server <-> Kerberos: The sever uses Kerberos for authentication, token validation and role-based access controls on all protected actions.

- Server -> Tallier: Server pseudonymises voter identities before releasing the ballots for tallying.

The system follows a zero-trust architecture where no client component inherently trusts any other. All state changing and sensitive read actions excluding /voters are authenticated and authorised by the server. This is in line with the common security ideals of “never trust always verify” (Anon-i, 2025).

Analysis of the Solution

Design rationale and trade-offs.

Confidentiality of ballot contents with RSA-OAEP. I chose this as it provides security against chosen-plain text attacks and breaking the encryption without the private key takes a large amount of computing power and time (Anon-h, n.d.). The voters encrypt their ballots with the RSA public key, which protects the contents of the ballot when at rest and when being transferred. The server only stores the cipher text and the tallier decrypts after the election. Key rotation occurs per election which is based on the key id this limits the impact of a key compromise. This protects against eavesdropping as it encrypts the ballot that is being stored and transferred. A trade off being that there is computational overhead for each vote, but it is simpler for distributing the key than symmetric encryption.

Ballot authentication with ECDSA over P-256. This was chosen due to the higher security for a given key length than RSA with less overhead (Miller, 2022). The client signs their cipher text and voter id with their private PEM. The server can verify this against their public key which prevents ballot forgery and binds the signature to the exact cipher text to protect against tampering or substitution. A downside to this is the per request cryptography and the extra work required by the registrar along with securely managing the private PEMs.

Post election decryption via Shamir secret sharing of d. The admin splits d into N shares with a threshold of T both of which are given by the user. Then after the election has closed the tallier can reconstruct d only If they have enough shares to reach or exceed the threshold and using p and q which should be securely stored and transferred off band. Reduces the chances of insider risk as requires multiple insider threats. Does add overhead with distributing, tracking, and collecting shares between talliers but the improvement in security against insider threats is worth it.

Simple HTTP server with limited surface and failure closed defaults. Voting is disabled when the election is closed or public key is not set, and results are closed when voting is open. Results remain closed until the tallying is done, are returned, and voting is closed. This leaves the server with a small surface area for attacks. This protects against accidental disclosure or tallying while voting is still open keeping operations in order. This does have less build in defences due to its simplicity but allows for more focus on exposed endpoints.

Role-based authentication with HMAC tokens. I decided on this due to the simplicity and strength of HMAC as well as how fast it can be computed (Anon-f, 2025) to allow for minimal effect on users. Passwords are hashed with pbkdf2-hmac-sha256 with a per user salt and stored this is following along with OWASPs guidelines on password storage (Anon-g, 2025). Once a user has logged in, they are issued a HMAC token signed by a server secret which we are

assuming is stored securely. Endpoints on the server require specific roles to access them which is checked by the bearer token in the authorisation header. This protects against password disclosure at rest as the passwords and stored hashed. Unauthorised access to protected endpoints is also reduced by this and the tokens mac stops tampering. There is no server-side rotation schedule so the secret stays the same, but this can be changed manually. The server does not offer a revocation list either, but the lightweight style allows for simple implementation.

Deduplication during submission and tallying. The server rejects second ballots from the same key id which references the election and voter id enforcing a one vote policy. When the ballots get passed to the tallier for counting it does another deduplication check to find any that potentially bypassed this check and only counts the first vote. This stops double voting taking place which can affect integrity of the election and replaying of votes. A trade-off being extra processing to check twice but it is lightweight and effective.

File based key stores working under the assumption in actual implementation they would be securely stored. These stores maintain a separation of privileges; the registrar maintains and generates the signature keys and public keys for voters. The admin maintains the election and secret keys. Does allow for breaches if not stored securely this could be fixed by encryption or access controls.

Results persistence with append only file. Once a tally has been posted to the server it is timestamped and appended to a result .Json file. This allows for a basic audit trail of previous results with their key id and does not include any information that should not be private like who voted. This is not tamperproof so adding something like hashing could resolve this issue.

Signature verification binds to ciphertext not plaintext. These stops cut and paste attacks on signed names as the decrypt and counting only credits valid candidate names. So, if a valid signature is passed but the plaintext is invalid then the vote is ignored.

Per-election key rotation based on key id. This limits the effect if the RSA is compromised due to it being specific for only that one election. Meaning once it is exposed it is only valid until the next election, this is cheap to implement as the admin creates a new public key before a new election is opened.

Server-side logging for all actions that are security relevant which contains the role, user ID endpoint, method and other relevant data. This protects against repudiation by ensuring that all actions are traceable back to authenticated roles and users. Supports post incident investigations by providing a timeline and a detailed insight into what has happened. Logging improves accountability but increases the amount of sensitive data being stored. To mitigate this no sensitive client data is stored like ciphertext of votes only actions and other metadata. Also working under the assumption that the log is write once read only and securely stored with limited/restricted access on the server.

Pseudonymisation of voter IDs sent to the talliers from the server. This aligns with the GDPR pseudonymisation principles which limits the chance of exposing personal data (Anon-e, 2021). The talliers receive only the ciphertext and the pseudonym of the voter ID never their identity. This protects against information disclosure by preventing the talliers from learning which voter submitted which vote. It could also protect against threats and coercion by ensuring that the talliers do not learn of who voted for who. Minimal computational overhead for creating the

pseudonyms per ballot on the server side but no changes on client or tallier. The downside is the server still learns the ciphertext and voter pairing, but they do not decrypt the ciphertext.

STRIDE threat modelling.

Spoofing –

1. The threat could be an adversary impersonating a voter. The scenario is an adversary submitting a ballot from a voter id not associated to them. This would affect the server as it would receive a fraudulent vote and the tallier as they would count said vote. The mitigation in place for this is ECDSA signature over the voter id and the cipher text verified against the registered public key. The voting endpoint is also role gated with a HMAC token with an expiry which authenticates the user voting. This works as an attacker without the voters' private key fails the signature verification on the server.
2. Another threat could be the impersonation of a privileged role like admin or tallier. For example, the attacker could open or close the election as the admin. This would affect the server and admin as opening would generate election keys, and the server would open endpoints. This is mitigated by the role gated endpoints using HMAC tokens as the token is verified before it allows access to the endpoints. This works as a caller without a valid token is rejected.

Tampering –

1. The threat could be modifying data in transit. For example, changing ballot payloads in transit from the voters to the server. This would affect the server and the tallier as they would get fraudulent votes. Signature verification takes place on the votes which prevents ciphertext or identity substitution as votes contain both. This works due to ECDSA detecting any mutation to either that are used to produce the signature.
2. A threat could be modifying credentials that are at rest. For example, an adversary could overwrite an admin password hash to a hash of their choosing. This would affect the authorisation of the server; this could be mitigated by signed HMAC manifests for these files which are checked and verified each login. This would work as any adjustment to the HMAC manifest would show and thus stop the attacker from being able to login.
3. Tampering with the log. An adversary could either modify the logs or delete relevant logs that cover their malicious activity. This damages the integrity of the log and removes the accountability of users. This could be mitigated by using write once read only policy (Sheldon, 2022) or append only to avoid tampering. To mitigate modification going unnoticed a hash chaining scheme (Yavuz, n.d.) could be implemented which would flag on changes/ deletions.

Repudiation –

1. Tallier denies a post result. This could occur when a tallier disputes the results after they have been published. This would affect the server and the logging components. This is mitigated by the results being appended with the key id of the election and a timestamp which is when the results were retrieved. This works because it is an append only record so it keeps an audit trail that can be viewed but not changed assuming proper access controls.
2. Voter denies a vote. For example, after voting the voter claims that they did not vote. This would affect the integrity of the election process. The mitigation for this which is based on a privacy first idea, is verifying ECDSA at submission but not keeping a log of who

they voted for. Only voter in possession of the correct private key can submit a valid ballot. The server does not retain any decrypted vote content and only during the election temporarily stores a mapping between voter ID and encrypted ballot. As a result, post-election non-repudiation is not enforced by design and repudiation is allowed to prioritise voter anonymity.

3. A privileged user denies performing an action. For example, an admin denies closing the election. This would affect the auditability of the system as there is no record of who performed what action, also affecting incident reconstruction. The mitigation in place is server-side logging which records the role, user ID, endpoint and method for all security relevant actions. This creates an audit trail that helps with forensic analysis as it can set a time and specific user to each action. In the current implementation it is not tamper resistant so it could be altered. This works as actions cannot be conducted without generating a log entry detailing what they did and who did it.

Information Disclosure –

1. The threat might be the reading of confidential ballot contents. For example, this could occur due to network eavesdropping by an adversary. This would affect the ballots and the privacy of the voters. The mitigation in place is RSA-OAEP encryption of the ballot payload between clients, talliers and servers. This works as without d the ciphertext reveals nothing to the attacker.
2. The potential read of sensitive server data is a threat. This could be by a server compromise by an attacker. This would affect the stored ballots list but is mitigated by RSA-OAEP encryption of ballots and only the ciphertext is stored on the server. The ballots endpoint is also restricted to just the tallier which requires authentication and the election to be closed. This works because the ballots endpoint limits who can obtain the cipher text and without d the ciphertext reveals nothing to the attacker.
3. Reading of sensitive decrypted information at the tallier. For example, a compromised tallier could learn which voter submitted votes for specific candidates. This would affect the ballot secrecy and lead to potential coercion or retaliation on users breaking privacy and integrity of system. To mitigate this the server replaces the voter IDs with a randomly generated per election HMAC derived pseudonym before passing on to the tallier. This prevents the tallier from independently linking the votes to the specific voters, as the pseudonym is deterministic only within the election and unlink able without the server secret.

Denial of Service –

1. The threat would be an attempted exhaustion of server resources. For example, a flood of votes with requests or large bodies. This would affect the server as it would have to manage these requests. This is mitigated in part by the status of the server having to be open, the content type being limited to Json and signatures being verified. Optional additional mitigations could be rate limits or body size caps to reduce the chances of these attacks. This works due to fast rejections based on requirements that may not be known to the attackers.
2. Another threat would be exhaustion of server resources but this time by replaying duplicate votes to the server. This would have to be managed by the server, but duplicates are supressed by the duplicate check of key id and the voter id allowing for early drops. This works as the duplicates are dropped before they are decrypted or tallied reducing work.

Elevation of Privileges –

1. Threat would be performing actions that are not allowed by assigned role. For example, someone who is not an admin opening or closing the election. The main effect here would be on the server and admin as the state of election will be changed without the admin. Mitigation in place is authentication and role checks on the admin roles like opening and closing. This works as there is complete mediation on endpoints as they enforce roles verified by authorisation tokens.
2. Another threat would be performing actions that are out of order. For example, the tally being completed whilst the voting is still open. The effects here would be on the server and the ballots returned as they would potentially be incomplete. The mitigation in place is that the tally endpoint is only available when the status of the election is set to closed. This works as the mediation is enforced by the state machine itself.

Attack ability assessment.

Replay attack – A replay attack would be reposting the same user's vote or multiple users' votes that have already been sent. An adversary would need to capture a valid encrypted vote in transit or in memory to perform this. Currently in place the server drops duplicates based on the election's key id and the voters unique id. This occurs at submission, so they will not get decrypted and processed and the key id is only used for one election. If they manage to get past this then the tallier de-duplicates again and only keep the first vote before counting. The addition of logging also creates an audit trail that can reveal potential replay attacks that are unsuccessful and are dropped. The set is kept in memory which could be a risk due to potential loss as it will not be remembered. A mitigation to this could be keeping a record that could be used to rebuild the set, and the tallier would still de-duplicate this. In comparison to a databased backed system, they would survive restarts and would be inherently harder to replay attack, but they add extra database complexity and overhead. Another mitigation to replay attacks could be signature-based authentication (Anon-k, 2025) although this would increase operational overhead.

Man-in-middle – A man-in-middle attack that could take place would be stealing passwords or bearer tokens by sniffing the authorisation header. An adversary would need to position themselves along the network path to be able to intercept unencrypted HTTP traffic. The expiration of tokens is an existing control against this as it means intercepted tokens cannot be reused indefinitely, they are limited and strict timings can reduce the harm. RSA-OAEP keeps the ballot securely encrypted even if they are intercepted. This does mean that the tokens and passwords are repayable during the time-to-live. A mitigation to this would be using TLS for secure end-to-end confidentiality and integrity for credentials. In comparison to JWT over TLS systems, which combine transport layer security with token-based authentication to avoid main-in-middle attacks, but they increase the overhead and set up.

Denial of service – A denial of service attack could be flooding the /vote endpoint with oversized Json files to exhaust resources on the server. An adversary would only require access to the exposed endpoint /vote to flood it. Early checks are in place for incoming Json files like the status of voting must be open; signature of the voters must be verified, and duplicate votes cannot be sent in. No body cap sizes are in place or rate limits, so this is still an issue. To avoid this body cap sizes could be put into place and rate limits per IP (Anon-j, n.d.), or role could be added based on what they are required to send. In comparison to servers that implement web

application firewalls which filter and monitor incoming and outgoing packets would limit denial of service attacks but require configuration overhead.

Eavesdropping – An eavesdropping attack could be by passive network capture of packets sent. An adversary would have to have passive access to the network to be able to monitor and capture the network traffic. Client encrypts votes with RSA-OAEP before they are sent over the network and only the cipher text with the voter id is sent to the server. When the votes are sent to the tallier even the voter IDs are pseudonyms so this can only occur once. Residual risk is that passwords and tokens are readable over the network if we are not implementing TLS. To mitigate this risk, TLS could be implemented to secure the transmission (Haastrup, 2025). Token expiration times could also be reduced to limit the damage once exposed. In comparison to services which offer TLS and token binding which limit credential exposure from packets in transit higher overhead but added security.

Ransomware – Ransomware attacks could be either the encryption or deletion of files crucial to the functioning of the system for example auth, signature keys and election keys. An adversary would need to be able to gain write access to the server or the file system that stores the relevant data. There are no current controls to stop this other than the check that they exist. To mitigate this the assumption of secure off band storage could also have off host backups which are updated regularly and can be changed to earlier versions. Write once read only access controls should be implemented to stop deletion and potential tampering too. In comparison to services which offer key management systems which are designed for generation, storage, and distribution of keys. This is a much more complex implementation but gives a greater security and auditability of keys which makes it harder for deletion of data.

Data manipulation – A data manipulation attack could be replacing the talliers password hash to one set by the attacker at rest. This would require the adversary to obtain file system write access and to know the locations of authorisation data at rest. The hashing function in place slows cracking of the hash but it does not protect against replacement. If the attacker is allowed to write to files, then they can change it without it being known. This could be mitigated by HMAC of the stored password hashes and on reading verifying the hash against the HMAC registry to avoid the data being manipulated. In comparison to systems that use a database backed with row check sums to trigger on attempted data manipulation more accurately.

Secure design methodology evaluation

Principle of Least privilege – The system follows this by having role-based endpoints which only allows certain roles to access these endpoints. For example, only authorised voters can vote, only authorised admins can open/ close elections or post the public key. The state of the server also gates endpoints which stops out of order operations like votes are only allowed when the election is open. The server stores minimal data on it which is the ciphertext ballots which trusts no one with plaintext except for the tallier when they have enough talliers to calculate d. The voter IDs passed to the talliers are pseudonyms which ensures that the talliers only learn what is required for tallying. The limitations of this in the current implementation are credentials and tokens are passed unencrypted which can allow adversaries to escalate their privileges. The file stores in the initial assumption are being trusted but if they are exposed and writeable then attackers can potentially swap password hashes and elevate privileges. Improvements I would make are TLS encryption for authorisation to stop credentials being exposed on transfer. HMAC signature check list for password hashes that can be compared against to verify passwords and ensure password hashes have not been tampered with. Potentially also

implementing an ECDSA challenge and response for the admin and tallier per requests and for logging in to void potential retries with same password or token.

Separation of Privilege – The system implements this by only decrypting once the threshold of talliers is met, it does this by Shamir's T-of-N on d secret sharing which means that no single tallier can decrypt. The main duties are split on the system for example the admin generates and posts the public keys and controls the state of the election. The tallier decrypts and posts the results to the server and the server itself never holds the d required to decrypt. The registrar registers voters to the systems and generates their ECDSA private PEMs and updates the public keys file. Submission and counting are separated; the server accepts only signed cipher texts from verified voter ids and deduplicates. The tallier then takes these de-duplicates again then decrypts and counts in a different step. The limitations of this are p and q are stored together in the same file and are not separated meaning one insider could get them both. Registrar is a single operator meaning one actor can add voters which could lead to forged votes. The admin controls the open and close alone and does not require dual controls for state changes. To improve the system the custody of p and q could be split between two or encrypted with a second key which requires multiple shares. Admin state changes could require two or more users to open or close the election this could be done with two approvals being sent or two EDCSA signatures. The same change could be made to the registrar to make adding voters require two or more users even if it is taken from a defined list having multiple check is better.

Overall Assessment

Case Study – Kademlia /Kad DHT poisoning attack on eMule

Architecture

Kademlia is a distributed hash table protocol which is used in decentralised peer-to-peer networks to serve data without the need for a centralised entity (Singh, 2022). Kademlia does this by having a network of co-operating nodes with unique node-ids that communicate with each other. When a node in the network needs a block of data it searches nodes closest to the key associated with that data, this allows for fast retrieval (Anon-c, 2010). Nodes can locate which nodes store the required data by looking at their routing tables which store contacts and relaying queries to closest known contacts (Anon-d, 2025). The Kad network implements the Kademlia protocol and maintains large routing tables which allows for keyword lookups for file sharing which is used by eMule (Anon-b, 2010).

The Attack

The preparation phase starts by an attacker setting up n hosts and distributing their own table to each whilst also giving each node their own ID. A malicious node is also set up on each computer. When the nodes then join the Kad network, they find their neighbours and query until they cannot find any new node, or they run out of available bandwidth. Malicious nodes also hijack pointers in honest nodes claiming to be honest nodes (Wang, 2009). This is possible due to Kads lack of ID authentication and the permission for nodes to set their own ID's.

The Execution phase exploits a Kademlia weakness to make Kad look ups fail when a malicious node is queried. This cannot be done simply by dropping the lookups due to how Kademlia manages non-responders as they get evicted after a missed KADEMLIA_REQ. So, they instead target the neighbour selection of nodes which chooses the closest contacts to the key that is

being queried. The malicious nodes spoof or choose node IDs and return those contacts which appear closer than honest nodes hijacking the query path. Once this is done the attacker controls the subsequent hops and can cause the lookup to fail and be terminated due to the termination conditions of the DHT (Wang, 2009).

Response

In response to this attack uses on the Kad network implemented their own changes to their nodes to counter this. One of these changes is flooding protection that limits the number of messages processed from each IP address this can protect against DDoS attacks by limiting the flooding of normal control messages (Koo, 2012). This leads on to Kad nodes limiting entries in their routing tables based on the IP address received by and the /24 subnet which stops singular malicious nodes re-writing and poisoning whole routing tables. EMule themselves were constantly developing and adapting their system (Mysicka, 2006) an example of this is the implementation of protocol obfuscation by encrypting packets this makes data appear random to adversaries so they can't recognise eMule is using that connection (Anon-a, 2006). Hashing of keyworks which are used in queries was also implemented which made It harder to poison the route of the node, but this could be circumvented by a reverse hash of the keyword via a dictionary attack, so its protection became limited.

Reflection

All the amendments that have come about due to the discovery of these attacks are not in place across all nodes on the Kad network. As eMule is a decentralised peer-to-peer system there is no central authority to enforce security policies. This means that although certain nodes may implement security features that either circumvent or make these attacks difficult other nodes may not causing lookups to suffer. This is a large downside to decentralised peer-to-peer systems as each peer in the network can have dire consequences for others, but that same separation is also key to the fundamental principles of such systems.

Prevention and Mitigation

To prevent this attack the principle of least privilege could be used to not necessarily distrust nodes but to verify them (Wang, 2009) before you allow them to give you contacts or nodes for your routing table. This would stop malicious nodes from being able to poison your routing tables without verification first, making an attack more difficult. Another mitigation that could be implemented is authentication systems to register and authorise nodes (Locher, 2011) this would stop unauthenticated nodes from being able to affect honest nodes. The problem with this is that a centralised authentication system defeats the purpose of peer-to-peer systems, which makes the implementation more challenging whilst still maintaining a separation of privileges between nodes. A certificate authority could be used to register trusted nodes, which can vouch for other nodes which could be considered as well as the proximity to the key to mitigate this attack.

References

Anon-a, 2006. *Protocol Obfuscation*. [Online]
Available at: https://www.emule-project.com/home/perl/help.cgi?l=1&rm=show_topic&topic_id=848
[Accessed 7 11 2025].

Anon-b, 2010. *Introduction Kad*. [Online]
Available at: https://wiki.emule-web.de/Introduction_Kad
[Accessed 7 11 2025].

Anon-c, 2010. *Kademlia: A Design Specification*. [Online]
Available at: https://xlattice.sourceforge.net/components/protocol/kademlia/specs.html
[Accessed 7 11 2025].

Anon-d, 2025. *Distributed Hash Tables with Kademlia*. [Online]
Available at: https://www.geeksforgeeks.org/system-design/distributed-hash-tables-with-kademlia/
[Accessed 7 11 2025].

Anon-e, 2021. *Pseudonymization according to the GDPR*. [Online]
Available at: https://dataprivacymanager.net/pseudonymization-according-to-the-gdpr/
[Accessed 8 12 2025].

Anon-f, 2025. *Why HMAC Is Still a Must-Have for API Security in 2025*. [Online]
Available at: https://www.authgear.com/post/hmac-api-security
[Accessed 8 12 2025].

Anon-g, 2025. *Password Storage Cheat Sheet*. [Online]
Available at: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
[Accessed 8 12 2025].

Anon-h, n.d. *RSAES-OAEP Encryption Scheme,* Bedford: RSA Labratories.
Available at: https://www.inf.pucrs.br/calazans/graduate/TPVLSI_I/RSA-oaep_spec.pdf
[Accessed 8 12 2025].

Anon-i, 2025. *Zero Trust in action: Building a secure futur*. [Online]
Available at: https://www.microsoft.com/en-us/security/business/zero-trust
[Accessed 8 12 2025].

Anon-j, n.d. *Infrastructure support team mitigates DDoS with AWS WAF and rate limits*. [Online]
Available at: https://www.adaptavist.com/case-studies/infrastructure-support-team-mitigates-ddos-with-aws-waf-and-rate-limits
[Accessed 8 12 2025].

Anon-k, 2025. *Threats Your Guide to Replay Attacks*. [Online]
Available at: https://www.packetlabs.net/posts/a-guide-to-replay-attacks-and-how-to-defend-against-them/
[Accessed 8 12 2025].

Haastrup, A., 2025. *API security best practices: tips to protect your data in transit.* [Online]
Available at: https://www.cerbos.dev/blog/api-security-best-practices
[Accessed 8 12 2025].

Koo, H., 2012. *A DDoS attack by flooding normal control messages in Kad P2P networks,* s.l.: Ajou University.
Available at: https://ieeexplore.ieee.org/document/6174645/authors#authors
[Accessed 8 11 2025]

Locher, T., 2011. *eDonkey & eMule's Kad: Measurements & Attacks,* s.l.: IBM.
Available at: https://tik-db.ee.ethz.ch/file/83c0bccd6d7710bf1bea667f71040ac8/
[Accessed 8 11 2025]

Miller, Z., 2022. *How to evaluate and use ECDSA certificates in AWS Certificate Manager.* [Online]
Available at: https://aws.amazon.com/blogs/security/how-to-evaluate-and-use-ecdsa-certificates-in-aws-certificate-manager/
[Accessed 8 12 2025].

Mysicka, D., 2006. *Reverse Engineering of eMule - An Analysis of the Implementation of Kademlia in eMule,* s.l.: Swiss Federal Institute of Technology Zurich.
Available at: https://pages.di.unipi.it/ricci/emule_reverse_report-Kademlia.pdf
[Accessed 8 11 2025]

Sheldon, R., 2022. *WORM (write once, read many).* [Online]
Available at: https://www.techtarget.com/searchstorage/definition/WORM-write-once-read-many
[Accessed 8 12 2025].

Singh, U., 2022. *Kademlia Distributed Hash Tables.* [Online]
Available at: https://singhuddeshyaofficial.medium.com/kademlia-protocol-in-distributed-hash-tables-ad95d17e64ba
[Accessed 7 11 2025].

Wang, P., 2009. *Attacking the Kad Network,* s.l.: University of Minneapolis.
Available at: https://www-users.cse.umn.edu/~hoppernj/kad_attack_securecomm.pdf
[Accessed 8 11 2025]

Yavuz, A. A., n.d. *Efficient, Compromise Resilient and Append-only Cryptographic Schemes for Secure Audit Logging,* Raleigh: North Carolina State University.
Available at: https://fc12.ifca.ai/pre-proceedings/paper_23.pdf
[Accessed 8 12 2025].